

Experiment ‘p_14_4_merge_sort_lexicographic_seq_RERUN-1’ Results

June 6, 2026

Experiment outcome: FAILURE_VERIFY

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 4

Hard fails (resolution): 1

Soft fails (resolution): 0

Verification attempts: 3

Problem Specification

Problem name: p_14_4_merge_sort_lexicographic_seq_RERUN-1

Natural language statement: Write a method that implements the merge sort algorithm to sort an array of strings in lexicographic order.

Method signature: p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)

Ensures

- forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
- multiset(inputs) == multiset(sorted)

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that implements the merge sort algorithm to sort an array of strings in
↪ lexicographic order.

The signature should be:

p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<string>)

The method should respect the following contract:

ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j], ensures multiset
↪ (inputs) == multiset(sorted)

Produce and show only the Dafny body of this method, including the curly braces that
↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
↪ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkWEbVUkRFfgUwn4s5AHwwE6JVHM

Model: gpt-5.5-2026-04-23

Created at: 1780749262

Finish reason: stop

Usage: 338 tokens in, and 5571 tokens out

```
{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;
  assert inputs == inputs[..mid] + inputs[mid..];

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
  var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

  var i := 0;
  var j := 0;
  sorted := [];

  while i < |left| || j < |right|
    invariant 0 <= i <= |left|
```

```

invariant 0 <= j <= |right|
invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
invariant forall k :: i <= k < |left| ==> |sorted| == 0 || sorted[|sorted|-1] <= left[
  ↪ k]
invariant forall k :: j <= k < |right| ==> |sorted| == 0 || sorted[|sorted|-1] <=
  ↪ right[k]
decreases |left| - i + |right| - j
{
  if i < |left| && (j == |right| || left[i] <= right[j]) {
    var oldSorted := sorted;
    var oldI := i;
    var x := left[oldI];

    assert forall a | 0 <= a < |oldSorted| :: oldSorted[a] <= x by {
      if |oldSorted| > 0 {
        assert oldSorted[|oldSorted|-1] <= x;
        if a < |oldSorted| - 1 {
          assert oldSorted[a] <= oldSorted[|oldSorted|-1];
        }
      }
    }

    sorted := oldSorted + [x];
    i := oldI + 1;

    assert left[..i] == left[..oldI] + [x];
    assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
    assert multiset(left[..i]) == multiset(left[..oldI]) + multiset([x]);

    assert forall a, b | 0 <= a < b < |sorted| :: sorted[a] <= sorted[b] by {
      if b < |oldSorted| {
        assert sorted[a] == oldSorted[a];
        assert sorted[b] == oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert sorted[b] == x;
        assert oldSorted[a] <= x;
      }
    }

    assert forall k | i <= k < |left| :: sorted[|sorted|-1] <= left[k] by {
      assert sorted[|sorted|-1] == x;
      assert oldI < k;
      assert left[oldI] <= left[k];
    }

    assert forall k | j <= k < |right| :: sorted[|sorted|-1] <= right[k] by {
      assert sorted[|sorted|-1] == x;
      if j < |right| {
        assert x <= right[j];
      }
    }
  }
}

```

```

    if j < k {
      assert right[j] <= right[k];
    }
  }
} else {
  assert j < |right|;
  var oldSorted := sorted;
  var oldJ := j;
  var x := right[oldJ];

  if i < |left| {
    assert !(left[i] <= right[oldJ]);
    assert right[oldJ] <= left[i];
  }

  assert forall a | 0 <= a < |oldSorted| :: oldSorted[a] <= x by {
    if |oldSorted| > 0 {
      assert oldSorted[|oldSorted|-1] <= x;
      if a < |oldSorted| - 1 {
        assert oldSorted[a] <= oldSorted[|oldSorted|-1];
      }
    }
  }

  sorted := oldSorted + [x];
  j := oldJ + 1;

  assert right[..j] == right[..oldJ] + [x];
  assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
  assert multiset(right[..j]) == multiset(right[..oldJ]) + multiset([x]);

  assert forall a, b | 0 <= a < b < |sorted| :: sorted[a] <= sorted[b] by {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert sorted[b] == x;
      assert oldSorted[a] <= x;
    }
  }

  assert forall k | j <= k < |right| :: sorted[|sorted|-1] <= right[k] by {
    assert sorted[|sorted|-1] == x;
    assert oldJ < k;
    assert right[oldJ] <= right[k];
  }

  assert forall k | i <= k < |left| :: sorted[|sorted|-1] <= left[k] by {
    assert sorted[|sorted|-1] == x;
  }

```

```

    if i < |left| {
      assert x <= left[i];
      if i < k {
        assert left[i] <= left[k];
      }
    }
  }
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(inputs[..mid]) == multiset(left);
assert multiset(inputs[mid..]) == multiset(right);
assert multiset(inputs) == multiset(sorted);
}

```

Program → GenAI

When using dafny resolve, the below error is emitted and resolve fails:

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(38,13): Error:

↪ unresolved identifier: a

```

|
38 | if a < |oldSorted| - 1 {
| ^

```

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(39,29): Error:

↪ unresolved identifier: a

```

|
39 | assert oldSorted[a] <= oldSorted[|oldSorted|-1];
| ^

```

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(52,11): Error:

↪ unresolved identifier: b

```

|
52 | if b < |oldSorted| {
| ^

```

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(53,24): Error:

↪ unresolved identifier: a

```

|
53 | assert sorted[a] == oldSorted[a];
| ^

```

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(53,40): Error:

↪ unresolved identifier: a

```

|
53 | assert sorted[a] == oldSorted[a];
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(54,24): Error:
  ↪ unresolved identifier: b
|
54 | assert sorted[b] == oldSorted[b];
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(54,40): Error:
  ↪ unresolved identifier: b
|
54 | assert sorted[b] == oldSorted[b];
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(56,17): Error:
  ↪ unresolved identifier: b
|
56 | assert b == |oldSorted|;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(57,24): Error:
  ↪ unresolved identifier: b
|
57 | assert sorted[b] == x;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(58,27): Error:
  ↪ unresolved identifier: a
|
58 | assert oldSorted[a] <= x;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(64,22): Error:
  ↪ unresolved identifier: k
|
64 | assert oldI < k;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(65,34): Error:
  ↪ unresolved identifier: k
|
65 | assert left[oldI] <= left[k];
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(72,17): Error:
  ↪ unresolved identifier: k
|
72 | if j < k {
| ^

```

```

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(73,37): Error:
    ↪ unresolved identifier: k
    |
73 | assert right[j] <= right[k];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(91,13): Error:
    ↪ unresolved identifier: a
    |
91 | if a < |oldSorted| - 1 {
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(92,29): Error:
    ↪ unresolved identifier: a
    |
92 | assert oldSorted[a] <= oldSorted[|oldSorted|-1];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(105,11): Error:
    ↪ unresolved identifier: b
    |
105 | if b < |oldSorted| {
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(106,24): Error:
    ↪ unresolved identifier: a
    |
106 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(106,40): Error:
    ↪ unresolved identifier: a
    |
106 | assert sorted[a] == oldSorted[a];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(107,24): Error:
    ↪ unresolved identifier: b
    |
107 | assert sorted[b] == oldSorted[b];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(107,40): Error:
    ↪ unresolved identifier: b
    |
107 | assert sorted[b] == oldSorted[b];
    | ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(109,17): Error:
    ↪ unresolved identifier: b

```

```

|
109 | assert b == |oldSorted|;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(110,24): Error:
  ↪ unresolved identifier: b
|
110 | assert sorted[b] == x;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(111,27): Error:
  ↪ unresolved identifier: a
|
111 | assert oldSorted[a] <= x;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(117,22): Error:
  ↪ unresolved identifier: k
|
117 | assert oldJ < k;
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(118,36): Error:
  ↪ unresolved identifier: k
|
118 | assert right[oldJ] <= right[k];
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(125,17): Error:
  ↪ unresolved identifier: k
|
125 | if i < k {
| ^

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(126,35): Error:
  ↪ unresolved identifier: k
|
126 | assert left[i] <= left[k];
| ^

28 resolution/type errors detected in program.dfy

Correct the error in code by altering only the method body. Produce and show only the
  ↪ Dafny body, including the curly braces that surround it. Do not show the signature
  ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

```

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkY1prnS9J0xBrtc7nZhhrdYOa0G

Model: gpt-5.5-2026-04-23

Created at: 1780749373

Finish reason: stop

Usage: 4727 tokens in, and 1961 tokens out

```
{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;
  assert inputs == inputs[..mid] + inputs[mid..];

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
  var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

  var i := 0;
  var j := 0;
  sorted := [];

  while i < |left| || j < |right|
    invariant 0 <= i <= |left|
    invariant 0 <= j <= |right|
    invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
    invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
    invariant forall k :: i <= k < |left| ==> |sorted| == 0 || sorted[|sorted|-1] <= left[
      ↪ k]
    invariant forall k :: j <= k < |right| ==> |sorted| == 0 || sorted[|sorted|-1] <=
      ↪ right[k]
    decreases |left| - i + |right| - j
  {
    if i < |left| && (j == |right| || left[i] <= right[j]) {
      var oldSorted := sorted;
      var oldI := i;
      var x := left[oldI];

      forall a | 0 <= a < |oldSorted|
        ensures oldSorted[a] <= x
      {
        if |oldSorted| > 0 {
          assert oldSorted[|oldSorted|-1] <= x;
          if a < |oldSorted| - 1 {
            assert oldSorted[a] <= oldSorted[|oldSorted|-1];
          }
        }
      }

      sorted := oldSorted + [x];
      i := oldI + 1;

      assert left[..i] == left[..oldI] + [x];
    }
  }
```

```

assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
assert multiset(left[..i]) == multiset(left[..oldI]) + multiset([x]);

forall a, b | 0 <= a < b < |sorted|
  ensures sorted[a] <= sorted[b]
{
  if b < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == oldSorted[b];
  } else {
    assert b == |oldSorted|;
    assert sorted[b] == x;
    assert oldSorted[a] <= x;
  }
}

forall k | i <= k < |left|
  ensures sorted[|sorted|-1] <= left[k]
{
  assert sorted[|sorted|-1] == x;
  assert oldI < k;
  assert left[oldI] <= left[k];
}

forall k | j <= k < |right|
  ensures sorted[|sorted|-1] <= right[k]
{
  assert sorted[|sorted|-1] == x;
  if j < |right| {
    assert x <= right[j];
    if j < k {
      assert right[j] <= right[k];
    }
  }
}
} else {
  assert j < |right|;
  var oldSorted := sorted;
  var oldJ := j;
  var x := right[oldJ];

  if i < |left| {
    assert !(left[i] <= right[oldJ]);
    assert right[oldJ] <= left[i];
  }

  forall a | 0 <= a < |oldSorted|
    ensures oldSorted[a] <= x
  {
    if |oldSorted| > 0 {
      assert oldSorted[|oldSorted|-1] <= x;
    }
  }
}

```

```

    if a < |oldSorted| - 1 {
      assert oldSorted[a] <= oldSorted[|oldSorted|-1];
    }
  }
}

sorted := oldSorted + [x];
j := oldJ + 1;

assert right[..j] == right[..oldJ] + [x];
assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
assert multiset(right[..j]) == multiset(right[..oldJ]) + multiset([x]);

forall a, b | 0 <= a < b < |sorted|
  ensures sorted[a] <= sorted[b]
{
  if b < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == oldSorted[b];
  } else {
    assert b == |oldSorted|;
    assert sorted[b] == x;
    assert oldSorted[a] <= x;
  }
}

forall k | j <= k < |right|
  ensures sorted[|sorted|-1] <= right[k]
{
  assert sorted[|sorted|-1] == x;
  assert oldJ < k;
  assert right[oldJ] <= right[k];
}

forall k | i <= k < |left|
  ensures sorted[|sorted|-1] <= left[k]
{
  assert sorted[|sorted|-1] == x;
  if i < |left| {
    assert x <= left[i];
    if i < k {
      assert left[i] <= left[k];
    }
  }
}
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;

```

```

    assert right[..j] == right;
    assert multiset(sorted) == multiset(left) + multiset(right);
    assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
    assert multiset(inputs[..mid]) == multiset(left);
    assert multiset(inputs[mid..]) == multiset(right);
    assert multiset(inputs) == multiset(sorted);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(93,8): Error:
    ↪ assertion might not hold
    |
93 | assert right[oldJ] <= left[i];
    | ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
 ↪ Dafny body, including the curly braces that surround it. Do not show the signature
 ↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkYhLDAPI3hQsXWOsATqD2sPrVVa

Model: gpt-5.5-2026-04-23

Created at: 1780749415

Finish reason: stop

Usage: 4904 tokens in, and 5092 tokens out

```

{
    if |inputs| <= 1 {
        sorted := inputs;
        return;
    }

    var mid := |inputs| / 2;
    assert 0 < mid < |inputs|;
    assert inputs == inputs[..mid] + inputs[mid..];

    var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
    var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

    var i := 0;
    var j := 0;
    sorted := [];

```

```

while i < |left| || j < |right|
  invariant 0 <= i <= |left|
  invariant 0 <= j <= |right|
  invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
  invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
  invariant forall k :: i <= k < |left| ==> |sorted| == 0 || sorted[|sorted|-1] <= left[
    ↪ k]
  invariant forall k :: j <= k < |right| ==> |sorted| == 0 || sorted[|sorted|-1] <=
    ↪ right[k]
  decreases |left| - i + |right| - j
{
  if i < |left| && (j == |right| || left[i] <= right[j]) {
    var oldSorted := sorted;
    var oldI := i;
    var x := left[oldI];

    forall a | 0 <= a < |oldSorted|
      ensures oldSorted[a] <= x
    {
      if |oldSorted| > 0 {
        assert oldSorted[|oldSorted|-1] <= x;
        if a < |oldSorted| - 1 {
          assert oldSorted[a] <= oldSorted[|oldSorted|-1];
        }
      }
    }

    sorted := oldSorted + [x];
    i := oldI + 1;

    assert left[..i] == left[..oldI] + [x];
    assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
    assert multiset(left[..i]) == multiset(left[..oldI]) + multiset([x]);

    forall a, b | 0 <= a < b < |sorted|
      ensures sorted[a] <= sorted[b]
    {
      if b < |oldSorted| {
        assert sorted[a] == oldSorted[a];
        assert sorted[b] == oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert sorted[b] == x;
        assert oldSorted[a] <= x;
      }
    }

    forall k | i <= k < |left|
      ensures sorted[|sorted|-1] <= left[k]
    {
      assert sorted[|sorted|-1] == x;

```

```

    assert oldI < k;
    assert left[oldI] <= left[k];
}

forall k | j <= k < |right|
  ensures sorted[|sorted|-1] <= right[k]
{
  assert sorted[|sorted|-1] == x;
  if j < |right| {
    assert x <= right[j];
    if j < k {
      assert right[j] <= right[k];
    }
  }
}
} else {
  assert j < |right|;
  var oldSorted := sorted;
  var oldJ := j;
  var x := right[oldJ];

  if i < |left| {
    assert !(left[i] <= right[oldJ]);
    assert left[i] <= right[oldJ] || right[oldJ] <= left[i];
    assert right[oldJ] <= left[i];
  }

  forall a | 0 <= a < |oldSorted|
    ensures oldSorted[a] <= x
  {
    if |oldSorted| > 0 {
      assert oldSorted[|oldSorted|-1] <= x;
      if a < |oldSorted| - 1 {
        assert oldSorted[a] <= oldSorted[|oldSorted|-1];
      }
    }
  }
}

sorted := oldSorted + [x];
j := oldJ + 1;

assert right[..j] == right[..oldJ] + [x];
assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
assert multiset(right[..j]) == multiset(right[..oldJ]) + multiset([x]);

forall a, b | 0 <= a < b < |sorted|
  ensures sorted[a] <= sorted[b]
{
  if b < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == oldSorted[b];
  }
}

```

```

    } else {
      assert b == |oldSorted|;
      assert sorted[b] == x;
      assert oldSorted[a] <= x;
    }
  }

forall k | j <= k < |right|
  ensures sorted[|sorted|-1] <= right[k]
{
  assert sorted[|sorted|-1] == x;
  assert oldJ < k;
  assert right[oldJ] <= right[k];
}

forall k | i <= k < |left|
  ensures sorted[|sorted|-1] <= left[k]
{
  assert sorted[|sorted|-1] == x;
  if i < |left| {
    assert x <= left[i];
    if i < k {
      assert left[i] <= left[k];
    }
  }
}
}
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(inputs[..mid]) == multiset(left);
assert multiset(inputs[mid..]) == multiset(right);
assert multiset(inputs) == multiset(sorted);
}

```

Program → GenAI

When using dafny verify, the below error is emitted and verify fails:

```

problem_data/p_14_4_merge_sort_lexicographic_seq_RERUN-1/program.dfy(93,8): Error:
  ↪ assertion might not hold
  |
93 | assert left[i] <= right[oldJ] || right[oldJ] <= left[i];
  | ~~~~~

```

Dafny program verifier finished with 1 verified, 1 error

Correct the error in code by altering only the method body. Produce and show only the
↪ Dafny body, including the curly braces that surround it. Do not show the signature
↪ nor contract. You must not use 'assume' nor 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-DnkaZnBHcZDryGtvE1tj8jEKz166v

Model: gpt-5.5-2026-04-23

Created at: 1780749531

Finish reason: stop

Usage: 6537 tokens in, and 4065 tokens out

```
{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;
  assert inputs == inputs[..mid] + inputs[mid..];

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
  var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

  var i := 0;
  var j := 0;
  sorted := [];

  while i < |left| || j < |right|
    invariant 0 <= i <= |left|
    invariant 0 <= j <= |right|
    invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
    invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
    invariant forall k :: i <= k < |left| ==> |sorted| == 0 || sorted[|sorted|-1] <= left[
      ↪ k]
    invariant forall k :: j <= k < |right| ==> |sorted| == 0 || sorted[|sorted|-1] <=
      ↪ right[k]
    decreases |left| - i + |right| - j
  {
    if i < |left| && (j == |right| || !(right[j] < left[i])) {
      var oldSorted := sorted;
      var oldI := i;
      var x := left[oldI];

      if j < |right| {
        assert !(right[j] < x);
        assert x <= right[j];
      }
    }
  }
```



```

}

forall a | 0 <= a < |oldSorted|
  ensures oldSorted[a] <= x
{
  if |oldSorted| > 0 {
    assert oldSorted[|oldSorted|-1] <= x;
    if a < |oldSorted| - 1 {
      assert oldSorted[a] <= oldSorted[|oldSorted|-1];
    }
  }
}

sorted := oldSorted + [x];
i := oldI + 1;

assert left[..i] == left[..oldI] + [x];
assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
assert multiset(left[..i]) == multiset(left[..oldI]) + multiset([x]);

forall a, b | 0 <= a < b < |sorted|
  ensures sorted[a] <= sorted[b]
{
  if b < |oldSorted| {
    assert sorted[a] == oldSorted[a];
    assert sorted[b] == oldSorted[b];
  } else {
    assert b == |oldSorted|;
    assert sorted[b] == x;
    assert oldSorted[a] <= x;
  }
}

forall k | i <= k < |left|
  ensures sorted[|sorted|-1] <= left[k]
{
  assert sorted[|sorted|-1] == x;
  assert oldI < k;
  assert left[oldI] <= left[k];
}

forall k | j <= k < |right|
  ensures sorted[|sorted|-1] <= right[k]
{
  assert sorted[|sorted|-1] == x;
  if j < |right| {
    assert x <= right[j];
    if j < k {
      assert right[j] <= right[k];
    }
  }
}

```

```

    }
  } else {
    assert j < |right|;
    var oldSorted := sorted;
    var oldJ := j;
    var x := right[oldJ];

    if i < |left| {
      assert right[oldJ] < left[i];
      assert right[oldJ] <= left[i];
    }

    forall a | 0 <= a < |oldSorted|
      ensures oldSorted[a] <= x
    {
      if |oldSorted| > 0 {
        assert oldSorted[|oldSorted|-1] <= x;
        if a < |oldSorted| - 1 {
          assert oldSorted[a] <= oldSorted[|oldSorted|-1];
        }
      }
    }
  }

  sorted := oldSorted + [x];
  j := oldJ + 1;

  assert right[..j] == right[..oldJ] + [x];
  assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
  assert multiset(right[..j]) == multiset(right[..oldJ]) + multiset([x]);

  forall a, b | 0 <= a < b < |sorted|
    ensures sorted[a] <= sorted[b]
  {
    if b < |oldSorted| {
      assert sorted[a] == oldSorted[a];
      assert sorted[b] == oldSorted[b];
    } else {
      assert b == |oldSorted|;
      assert sorted[b] == x;
      assert oldSorted[a] <= x;
    }
  }

  forall k | j <= k < |right|
    ensures sorted[|sorted|-1] <= right[k]
  {
    assert sorted[|sorted|-1] == x;
    assert oldJ < k;
    assert right[oldJ] <= right[k];
  }

```

```

    forall k | i <= k < |left|
      ensures sorted[|sorted|-1] <= left[k]
    {
      assert sorted[|sorted|-1] == x;
      if i < |left| {
        assert x <= left[i];
        if i < k {
          assert left[i] <= left[k];
        }
      }
    }
  }
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(inputs[..mid]) == multiset(left);
assert multiset(inputs[mid..]) == multiset(right);
assert multiset(inputs) == multiset(sorted);
}

```

Final Program

```

method p_14_4_merge_sort_lexicographic_seq(inputs: seq<string>) returns (sorted: seq<
  ↪ string>)
  ensures forall i, j :: 0 <= i < j < |sorted| ==> sorted[i] <= sorted[j]
  ensures multiset(inputs) == multiset(sorted)
{
  if |inputs| <= 1 {
    sorted := inputs;
    return;
  }

  var mid := |inputs| / 2;
  assert 0 < mid < |inputs|;
  assert inputs == inputs[..mid] + inputs[mid..];

  var left := p_14_4_merge_sort_lexicographic_seq(inputs[..mid]);
  var right := p_14_4_merge_sort_lexicographic_seq(inputs[mid..]);

  var i := 0;
  var j := 0;
  sorted := [];

  while i < |left| || j < |right|
    invariant 0 <= i <= |left|

```

```

invariant 0 <= j <= |right|
invariant forall a, b :: 0 <= a < b < |sorted| ==> sorted[a] <= sorted[b]
invariant multiset(sorted) == multiset(left[..i]) + multiset(right[..j])
invariant forall k :: i <= k < |left| ==> |sorted| == 0 || sorted[|sorted|-1] <= left[
  ↪ k]
invariant forall k :: j <= k < |right| ==> |sorted| == 0 || sorted[|sorted|-1] <=
  ↪ right[k]
decreases |left| - i + |right| - j
{
  if i < |left| && (j == |right| || !(right[j] < left[i])) {
    var oldSorted := sorted;
    var oldI := i;
    var x := left[oldI];

    if j < |right| {
      assert !(right[j] < x);
      assert x <= right[j];
    }

    forall a | 0 <= a < |oldSorted|
      ensures oldSorted[a] <= x
    {
      if |oldSorted| > 0 {
        assert oldSorted[|oldSorted|-1] <= x;
        if a < |oldSorted| - 1 {
          assert oldSorted[a] <= oldSorted[|oldSorted|-1];
        }
      }
    }

    sorted := oldSorted + [x];
    i := oldI + 1;

    assert left[..i] == left[..oldI] + [x];
    assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
    assert multiset(left[..i]) == multiset(left[..oldI]) + multiset([x]);

    forall a, b | 0 <= a < b < |sorted|
      ensures sorted[a] <= sorted[b]
    {
      if b < |oldSorted| {
        assert sorted[a] == oldSorted[a];
        assert sorted[b] == oldSorted[b];
      } else {
        assert b == |oldSorted|;
        assert sorted[b] == x;
        assert oldSorted[a] <= x;
      }
    }

    forall k | i <= k < |left|

```

```

    ensures sorted[|sorted|-1] <= left[k]
  {
    assert sorted[|sorted|-1] == x;
    assert oldI < k;
    assert left[oldI] <= left[k];
  }

forall k | j <= k < |right|
  ensures sorted[|sorted|-1] <= right[k]
  {
    assert sorted[|sorted|-1] == x;
    if j < |right| {
      assert x <= right[j];
      if j < k {
        assert right[j] <= right[k];
      }
    }
  }
} else {
  assert j < |right|;
  var oldSorted := sorted;
  var oldJ := j;
  var x := right[oldJ];

  if i < |left| {
    assert right[oldJ] < left[i];
    assert right[oldJ] <= left[i];
  }

forall a | 0 <= a < |oldSorted|
  ensures oldSorted[a] <= x
  {
    if |oldSorted| > 0 {
      assert oldSorted[|oldSorted|-1] <= x;
      if a < |oldSorted| - 1 {
        assert oldSorted[a] <= oldSorted[|oldSorted|-1];
      }
    }
  }
}

sorted := oldSorted + [x];
j := oldJ + 1;

assert right[..j] == right[..oldJ] + [x];
assert multiset(sorted) == multiset(oldSorted) + multiset([x]);
assert multiset(right[..j]) == multiset(right[..oldJ]) + multiset([x]);

forall a, b | 0 <= a < b < |sorted|
  ensures sorted[a] <= sorted[b]
  {
    if b < |oldSorted| {

```

```

    assert sorted[a] == oldSorted[a];
    assert sorted[b] == oldSorted[b];
  } else {
    assert b == |oldSorted|;
    assert sorted[b] == x;
    assert oldSorted[a] <= x;
  }
}

forall k | j <= k < |right|
  ensures sorted[|sorted|-1] <= right[k]
{
  assert sorted[|sorted|-1] == x;
  assert oldJ < k;
  assert right[oldJ] <= right[k];
}

forall k | i <= k < |left|
  ensures sorted[|sorted|-1] <= left[k]
{
  assert sorted[|sorted|-1] == x;
  if i < |left| {
    assert x <= left[i];
    if i < k {
      assert left[i] <= left[k];
    }
  }
}
}
}

assert i == |left|;
assert j == |right|;
assert left[..i] == left;
assert right[..j] == right;
assert multiset(sorted) == multiset(left) + multiset(right);
assert multiset(inputs) == multiset(inputs[..mid]) + multiset(inputs[mid..]);
assert multiset(inputs[..mid]) == multiset(left);
assert multiset(inputs[mid..]) == multiset(right);
assert multiset(inputs) == multiset(sorted);
}

```

Total Token Usage

Input tokens: 16506

Output tokens: 16689

Reasoning tokens: 10751

Sum of 'total tokens': 33195

Experiment Timings

Overall Experiment started at 1780749261717, ended at 1780749620181, lasting 358464ms (358.46 seconds)

Iteration #4 started at 1780749530902, ended at 1780749620181, lasting 89279ms (89.28 seconds)

Iteration #1 started at 1780749261718, ended at 1780749372569, lasting 110851ms (110.85 seconds)

Iteration #2 started at 1780749372569, ended at 1780749414530, lasting 41961ms (41.96 seconds)

Iteration #3 started at 1780749414530, ended at 1780749530897, lasting 116367ms (116.37 seconds)